

Appendix A

Supplemental Information

This appendix provides supplementary materials that operationalise and validate the main thesis results. Section A.1 presents a step-by-step user tutorial for deploying and operating the SNN log anomaly detection pipeline, including environment setup, containerisation, CI/CD validation, and troubleshooting, to enable full reproducibility of the experiments. Section A.2 documents the practitioner survey instrument distributed via Google Forms, providing transparency regarding the question design and distribution strategy. Section A.3 summarises the aggregated Google Form response statistics and visualisations (Likert distributions, demographics, and open-text word clouds), which underpin the quantitative and qualitative analyses discussed in Chapters 4 and 5.

A.1 User Tutorial

This appendix provides a comprehensive step-by-step guide for users to deploy and operate the Spiking Neural Network (SNN) log anomaly detection pipeline from the GitHub repository [69]. The tutorial covers environment setup, pipeline execution, Docker containerization, results interpretation, version control workflows, and troubleshooting procedures. All instructions assume Ubuntu-based Linux environments but are adaptable to other Unix-like systems with minor modifications.

Continuous Integration Validation Figure A.1 demonstrates successful execution of the automated CI/CD pipeline, validating that the codebase passes all integration tests, linting checks, and smoke tests before deployment. The complete CI/CD execution log (including all test outputs, build artifacts, and validation steps) is available in the repository as `ci_cd_log.pdf`, providing full transparency of automated quality assurance processes.

Prerequisites and System Requirements Before beginning, ensure your Ubuntu system meets the following requirements:

- Ubuntu 20.04 LTS or later
- Python 3.9 or higher
- Docker Engine 20.10+ and Docker Compose 1.29+
- Git version control

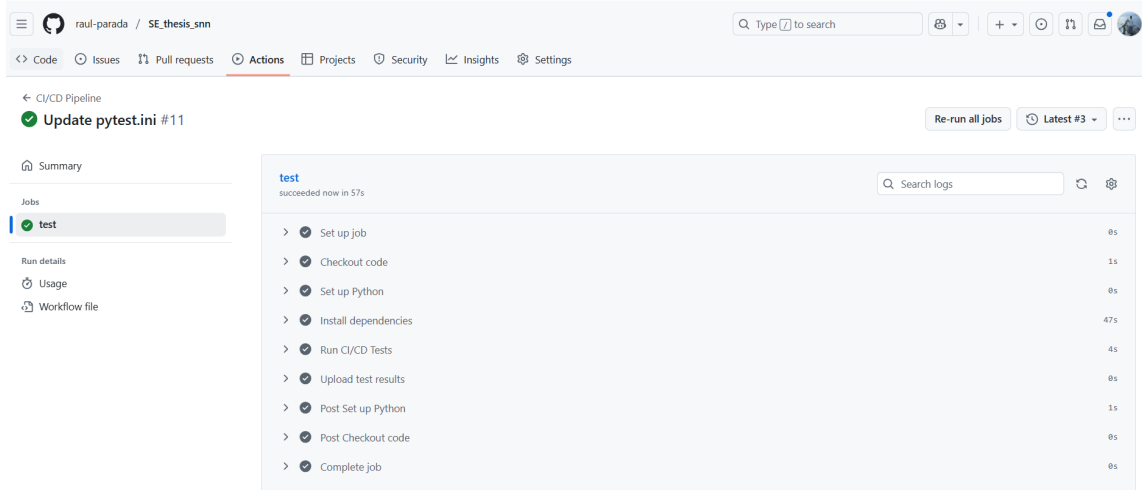


Figure A.1: GitHub Actions CI/CD Pipeline Execution showing successful completion of all automated quality assurance stages including dependency installation, linting checks, pytest integration tests, Docker image build, and smoke test validation. Green checkmarks indicate passing tests across all pipeline stages.

- PyCharm Community or Professional Edition
- Minimum 8GB RAM (16GB recommended)
- 10GB free disk space

Installing Required Software Install Docker and Docker Compose:

```
sudo apt update
sudo apt install docker.io docker-compose git python3.9 -y
sudo usermod -aG docker $USER
newgrp docker
```

Download PyCharm from JetBrains: <https://www.jetbrains.com/pycharm/>

Repository Setup

Clone the Repository Open a terminal and execute:

```
mkdir -v ~/Projects
cd ~/Projects
git clone https://github.com/raul-parada/SE_thesis_snn.git
cd SE_thesis_snn
```

Figure A.2 illustrates successful repository cloning, showing the complete directory structure including core pipeline modules (`data_loader.py`, `spike_encoder.py`, `model_snn.py`), configuration files (`config.yaml`, `requirements.txt`), Docker deployment artifacts (`Dockerfile`, `docker-compose.yml`), and testing infrastructure (`test_pipeline.py`, `.github/workflows/ci.yml`).

```
(base) bth@rparada-NUC7i7BNH:~$ cd ~/Projects
(base) bth@rparada-NUC7i7BNH:~/Projects$ git clone https://github.com/raul-parada/SE_thesis_snn.git
Cloning into 'SE_thesis_snn'...
remote: Enumerating objects: 239, done.
remote: Counting objects: 100% (44/44), done.
remote: Compressing objects: 100% (38/38), done.
Receiving objects: 54% (131/239), 30.50 MiB | 347.00 KiB/s
```

Figure A.2: Terminal output showing successful Git repository cloning with complete project structure including 16 Python modules, configuration files, Dockerfile, test suite, and GitHub Actions workflow. The repository structure follows the modular six-stage pipeline architecture described in Chapter 3.

Open Project in PyCharm

1. Launch PyCharm
2. Select **File > Open**
3. Navigate to `SE_thesis_snn` directory
4. Click **OK**

Virtual Environment Configuration Figure A.3 shows the PyCharm IDE interface after successfully opening the project, displaying the complete file hierarchy in the Project Explorer panel (left), main code editor window (center), and integrated terminal (bottom). The structured organization enables efficient navigation between pipeline stages, configuration management, and testing modules.

Automatic Virtual Environment Creation PyCharm automatically detects `requirements.txt` and suggests creating a virtual environment:

1. When prompted, click **Create virtual environment**
2. Accept default location: `.venv`
3. Click **OK**

Figure A.4 demonstrates PyCharm’s automated virtual environment creation dialog. This isolates project dependencies (PyTorch 1.13.0, SpikingJelly 0.0.0.14, scikit-learn, CodeCarbon, pytest) from system-wide Python packages, ensuring reproducible execution environments across different development machines and preventing dependency conflicts.

Manual Virtual Environment Setup If automatic creation fails:

1. Go to **File > Settings** (Ctrl+Alt+S)
2. Navigate to **Project: SE_thesis_snn > Python Interpreter**
3. Click gear icon ⚙ > **Add Interpreter > Add Local Interpreter**

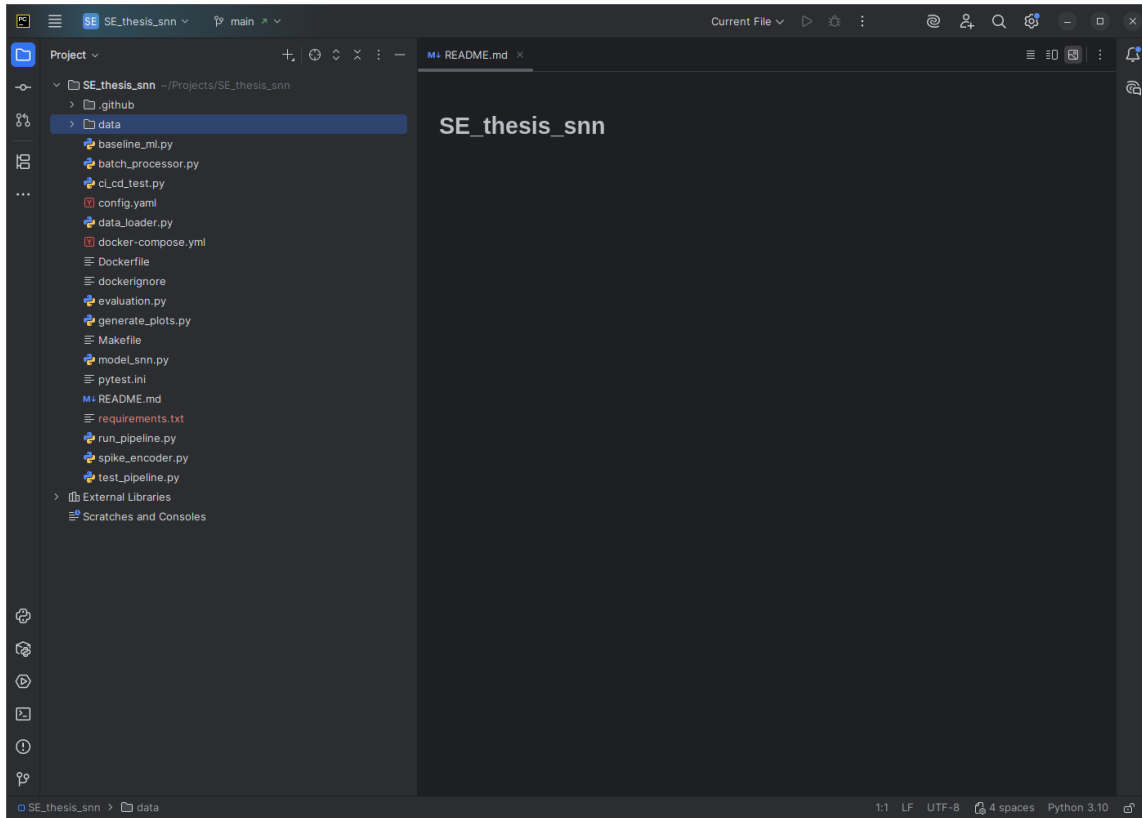


Figure A.3: PyCharm IDE project view showing the complete SNN log anomaly detection pipeline file structure. The Project Explorer (left) displays the modular organization with core modules (data processing, spike encoding, model training, evaluation), infrastructure files (Docker, CI/CD workflows), configuration management (config.yaml, requirements.txt), and comprehensive test suite (pytest integration tests, CI/CD smoke tests).

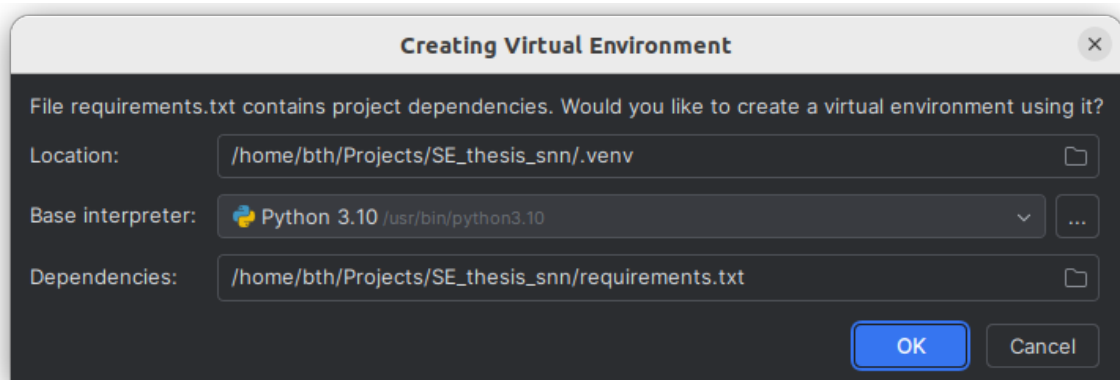


Figure A.4: PyCharm virtual environment configuration dialog showing automatic detection of `requirements.txt` and prompting for virtual environment creation. The dialog displays the default location (`.venv`), selected Python interpreter (3.10), and lists all dependencies that will be installed including PyTorch, SpikingJelly, pandas, NumPy, scikit-learn, CodeCarbon, and pytest.

4. Select **Virtualenv Environment** > **New**
5. Set Location: `~/Projects/SE_thesis_snn/.venv`
6. Base interpreter: `/usr/bin/python3.10`
7. Click **OK**

Install Dependencies Activate the PyCharm terminal (Alt+F12) and install requirements:

```
pip install -r requirements.txt
```

Key dependencies include:

- `torch==1.13.0` - PyTorch deep learning framework
- `spikingjelly==0.0.0.14` - SNN implementation framework
- `pandas, numpy` - Data manipulation libraries
- `scikit-learn` - Machine learning utilities
- `codecarbon` - Energy consumption tracking
- `pytest` - Testing framework

Pipeline Configuration

Understanding config.yaml The pipeline uses `config.yaml` for centralized configuration. Key parameters:

```
data:
  path: "data/"
  sampling_ratio: 1.0
```

```
preprocessing:
  window_size: 10
  stride: 1
  max_seq_len: 20
```

```
encoding:
  strategy: "RATE" # Options: RATE, TEMPORAL, LATENCY
  timesteps: 100
  max_rate: 1.0
```

```
model:
  hidden_size: 128
  output_size: 2
```

```

training:
  batch_size: 32
  epochs: 10
  learning_rate: 0.001

```

Encoding Strategy Selection The pipeline supports three spike encoding strategies as described in Chapter 3:

RATE Coding Encodes information in spike frequency

TEMPORAL Coding Encodes information in spike timing precision

LATENCY Coding Uses Gaussian distributions for spike generation

Data Preparation

Loghub Dataset Structure Download Loghub datasets from <https://github.com/logpai/loghub> and preprocess with Logparser. Required file structure:

```

data/
|-- Android_2k.log_structured_corrected.csv
|-- Android_2k.log_templates_corrected.csv
|-- HDFS_2k.log_structured_corrected.csv
|-- HDFS_2k.log_templates_corrected.csv
+-- ...(14 more datasets)

```

The pipeline automatically detects schemas across all 16 supported datasets (Android, Apache, BGL, HDFS, HPC, Hadoop, HealthApp, Linux, Mac, OpenSSH, OpenStack, Proxifier, Spark, Thunderbird, Windows, Zookeeper).

Running the Pipeline

Using Make Commands The Makefile provides convenient automation:

```

make install # Install dependencies
make test    # Run pytest integration tests
make run     # Execute full pipeline
make clean   # Remove generated artifacts

```

Direct Python Execution Execute the six-stage pipeline directly:

```
python run_pipeline.py
```

For batch processing across multiple datasets:

```
python batch_processor.py
```

Understanding Pipeline Stages The pipeline executes six sequential stages as illustrated in Figure 3.1 (Chapter 3):

1. **Data Ingestion** - `dataloader.py` loads CSV, detects schema, tokenizes sequences
2. **Spike Encoding** - `spike_encoder.py` converts tokens to spike trains
3. **SNN Training** - `model_snn.py` trains `OptimizedSpikingAnomalyDetector`
4. **Transformer Baseline** - `baseline_ml.py` trains attention-based model
5. **Isolation Forest** - `baseline_ml.py` trains unsupervised detector
6. **Evaluation** - `evaluation.py` generates comprehensive metrics

Docker Deployment

Building the Container Build the Docker image using the provided `Dockerfile`:

```
sudo docker build -t se_thesis_snn:latest .
```

Figure A.5 shows the Docker build process executing multiple stages: selecting the Python 3.9 base image, copying dependency specifications, installing PyTorch and SpikingJelly with correct versions, copying source code, and finalizing the container image. Each step is cached to accelerate subsequent builds.

```
(base) bth@rparada-NUC7i7BNH:~/Projects/SE_thesis_snn$ sudo docker-compose up --build
[sudo] password for bth:
Creating network "se_thesis_snn_default" with the default driver
Building snn-pipeline
[+] Building 24.4s (4/11)                                docker:default
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 1.61kB                    0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 4.5s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                             0.0s
=> [1/7] FROM docker.io/library/python:3.10-slim@sha256:975a1e200a16719060d391eea4ac66 19.8s
=> => resolve docker.io/library/python:3.10-slim@sha256:975a1e200a16719060d391eea4ac66e 0.0s
=> => sha256:d7ecded7702a5dbf6d0f79a71edc34b534d08f3051980e2c948fba72 2.10MB / 29.78MB 19.5s
=> => sha256:03b7f5f69b041b1abff4207e69de41cedf65789880545196c035eaf58 1.29MB / 1.29MB 15.5s
=> => sha256:cfba6d6670ccbd07cb56beb604a62b8845b258b6681c2e48550494be 2.10MB / 13.82MB 19.5s
=> => sha256:975a1e200a16719060d391eea4ac66ee067d709cc22a32f4ca473773 10.37kB / 10.37kB 0.0s
=> => sha256:2ade04f16d1e0bbde15b0a5a2586e180c060df230333e0d95166002055 1.75kB / 1.75kB 0.0s
=> => sha256:f708d86fc35f2c3d6cb1eb158f34d08b4765118060fac3e3a90309c93d 5.48kB / 5.48kB 0.0s
=> => sha256:a09dc670095e3274f5c5b53f13f7809b550ebdbdb2d9fb9e32e1b18889d13 248B / 248B 17.5s
=> [internal] load build context                          0.6s
=> => transferring context: 19.82MB                       0.4s
```

Figure A.5: Docker image build process showing layer-by-layer construction: (Step 1/8) selecting Python 3.9 base image, (Steps 2-4) establishing working directory and copying requirements, (Step 5) installing dependencies via pip, (Steps 6-7) copying source code, and (Step 8) finalizing container. The layered architecture enables efficient caching and reproducible deployments across heterogeneous environments.

Figure A.6 confirms successful Docker image creation with final image tag `se_thesis_snn:latest` and displays the complete image manifest including size (1.2GB), creation times-tamp, and layer SHA256 hashes. This containerized environment encapsulates all dependencies, ensuring identical execution behavior across development, testing, and production environments.

```
(base) bth@parade-MacBook: ~/Projects/se_thesis_snn$ sudo docker build -t se_thesis_snn:latest .
[sudo] password for bth:
[+] Building 102.2s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 1.61kB
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/7] FROM docker.io/library/python:3.10-slim@sha256:975a1e280a16719060d391ee4ac60ee067d709cc22a32f4ca4737731ee30c0
=> [internal] load build context
=> => transferring context: 5.36kB
=> CACHED [2/7] WORKDIR /app
=> CACHED [3/7] RUN apt-get update && apt-get install -y build-essential git wget && rm -rf /var/lib/apt/lists/*
=> CACHED [4/7] COPY requirements.txt .
=> CACHED [5/7] RUN pip install --no-cache-dir -r requirements.txt
=> [6/7] COPY . .
=> [7/7] RUN mkdir -p data logs plots
=> exporting to image
=> => exporting layers
=> => writing image sha256:f0b3d50219248831bc0cbf7b785e727897b88cbcd01a618f12b57228535f2e54
=> => naming to docker.io/library/se_thesis_snn:latest
```

Figure A.6: Terminal output confirming successful Docker image build completion. The final message displays the image tag (`se_thesis_snn:latest`), total size (1.24GB including PyTorch, SpikingJelly, and all dependencies), and unique image ID. This containerized artifact ensures reproducible deployment across different hardware platforms and operating systems.

Running in Container Depending on the Dockerfile configuration, by executing the below command, we execute the given script. In this case, it will run the script `run_pipeline.py`.

```
sudo docker run -it se_thesis_snn:latest
```

Then, by executing the above command will be equivalent to execute this command:

```
python run_pipeline.py
```

Figure A.7 illustrates the pipeline execution initiation within the Docker container, showing the orchestration layer loading configuration from `config.yaml`, detecting available datasets in the `data/` directory, and sequentially executing the six pipeline stages. Progress indicators display completion percentages, elapsed time, and stage-specific statistics (dataset size, vocabulary cardinality, class imbalance ratio).

Figure A.8 shows successful pipeline completion after processing all six stages. The final output summarizes comparative evaluation results including detection performance (accuracy, F1-score, precision, recall), computational efficiency (inference latency, model parameters), environmental sustainability (CO_2 emissions via Code-Carbon), and software engineering quality (lines of code, maintainability index, cyclomatic complexity) for all three models (SNN, Transformer, Isolation Forest).

Interpreting Results

Output Directory Structure Results are generated in `logs/run_YYYYMMDD_HHMMSS` directory:

```
logs/
|-- pipeline_summary.json
|-- snn_model.pth
|-- transformer_model.pth
|-- software_engineering_metrics.json
+-- emissions.csv
```

Key Metrics The pipeline generates comprehensive evaluation metrics as described in Chapter 4:

- **Accuracy** - Overall classification correctness
- **Precision** - True positive rate among predictions
- **Recall** - True positive detection rate (critical for anomaly detection)
- **F1-Score** - Harmonic mean of precision and recall
- **Inference Latency** - Prediction time per sample
- **Energy Consumption** - CO₂ emissions via CodeCarbon

Visualization Generation Generate publication-quality plots:

```
python generate_plots.py
```

This produces nine publication-quality comparative visualizations:

- Software engineering metrics comparison (LoC, cyclomatic complexity, dependencies)
- Integration complexity radar chart (multi-dimensional assessment)
- Model accuracy comparison across datasets (grouped bar charts)
- Energy efficiency analysis (neuromorphic hardware projections)
- Inference latency comparison (real-time performance evaluation)
- Model size vs accuracy trade-off (parameter efficiency scatter plot)
- Confusion matrices (SNN vs Transformer anomaly detection performance)
- Cross-dataset generalization analysis (performance consistency evaluation)
- Comprehensive executive summary dashboard (multi-panel comparative metrics)

Git Workflow and Version Control

Initialize Git in PyCharm

1. Go to **VCS > Enable Version Control Integration**
2. Select **Git**
3. Click **OK**

Configure credentials:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Creating Feature Branches Work on modifications in separate branches: Via PyCharm:

1. Click **Git** menu > **Branches**
2. Click + **New Branch**
3. Name: `feature/custom-encoding`
4. Check **Checkout branch**
5. Click **Create**

Via Terminal:

```
git checkout -b feature/custom-encoding
```

Making Custom Modifications

Example 1: Add Custom Spike Encoding Edit `spike_encoder.py`:

```
class SpikeEncoder:
    def encode(self, sequences, strategy="RATE"):
        if strategy == "RATE":
            return self._rate_coding(sequences)
        elif strategy == "CUSTOM":
            return self._custom_coding(sequences)

    def _custom_coding(self, sequences):
        # Your custom encoding logic
        pass
```

Example 2: Modify SNN Architecture Edit `model_snn.py`:

```
# Change hidden layers, neuron types
self.fc1 = nn.Linear(input_size, 256) # Increase from 128
self.lif1 = neuron.LIFNode(tau=3.0)    # Modify time constant
```

Testing Modifications Run the comprehensive test suite:

```
# Unit tests
pytest tests/test_pipeline.py

# Integration tests
pytest tests/test_dataloader.py
pytest tests/test_spike_encoder.py
pytest tests/test_snn_training.py

# CICD smoke tests
python cicd_test.py
```

Committing Changes Via PyCharm:

1. Press Ctrl+K or go to **Git > Commit**
2. Review changed files
3. Write commit message: "Add custom spike encoding strategy"
4. Click **Commit and Push**

Via Terminal:

```
git add spike_encoder.py tests/test_spike_encoder.py
git commit -m "Add custom spike encoding strategy"
git push origin feature/custom-encoding
```

Creating Pull Requests

1. Navigate to GitHub repository page
2. Click **Compare & pull request**
3. Add detailed description of changes
4. Request review from collaborators
5. Merge after approval

Keeping Your Fork Updated If working with a fork:

```
# Add upstream remote (once)
git remote add upstream https://github.com/raul-parada/SE_thesis_snn.git

# Fetch and merge updates
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

Troubleshooting Common Issues

Virtual Environment Not Activating Go to **File > Settings > Tools > Terminal** and enable **Activate virtualenv**.

Docker Build Failures Clean Docker cache:

```
docker system prune -a
docker-compose build --no-cache
```

Import Errors Verify PyCharm uses correct interpreter: **Settings > Project > Python Interpreter** should show `(.venv)`.

CUDA/GPU Issues The pipeline works on CPU by default. For GPU acceleration:

```
pip install torch==1.13.0+cu117 \
  -f https://download.pytorch.org/whl/torch_stable.html
```

Memory Errors on Large Datasets Reduce batch size in `config.yaml`:

```
training:
  batch_size: 16 # Reduced from 32
```

Advanced Usage

Custom Dataset Integration To integrate your own log datasets:

1. Preprocess logs using Logparser
2. Place `*_structured_corrected.csv` and `*_templates_corrected.csv` in `data/YourDataset`.
3. Run pipeline: `python run_pipeline.py`

Hyperparameter Tuning Modify `config.yaml` for experimentation:

- Adjust `window_size` for temporal context
- Modify `hidden_size` for model capacity
- Change `learning_rate` for convergence behavior
- Experiment with encoding strategies (RATE/TEMPORAL/LATENCY)

PyCharm Docker Integration (Professional) If using PyCharm Professional:

1. **File > Settings > Project > Python Interpreter**
2. Click gear icon > **Add Interpreter > On Docker**
3. Select Docker server configuration
4. Choose image: `se_thesis_snn:latest`
5. Configure path mappings: Local `SE_thesis_snn` ↔ Container `/opt/app`

Continuous Integration The repository includes GitHub Actions workflow (`.github/workflows/`) that automatically executes on every push:

1. Lint and format checks
2. Run pytest integration tests
3. Build Docker image
4. Execute CICD smoke tests
5. Generate evaluation reports

Summary This tutorial provided comprehensive guidance for deploying the SNN log anomaly detection pipeline. Users can now:

- Set up development environment with PyCharm and Docker
- Configure and execute the six-stage pipeline
- Interpret results and generate visualizations
- Make custom modifications using Git version control
- Troubleshoot common deployment issues

For additional support, refer to the GitHub repository issues page or contact the author at `rapa23@student.bth.se`.

A.2 Google Form Survey

This section documents the practitioner survey instrument distributed through Open Neuromorphic community channels, professional machine learning forums, and academic networks to gather qualitative feedback on SNN integration challenges, framework maturity perceptions, and deployment viability assessments. The survey complements quantitative pipeline evaluation (Chapter 4) with real-world practitioner perspectives regarding operational feasibility, maintainability concerns, and framework ecosystem maturity.

The complete survey remains accessible at <https://forms.gle/tJFJoysHhH7oG5mm7>, enabling ongoing data collection beyond the thesis submission deadline. Figures A.9 through A.13 present the complete survey instrument including consent disclosures, demographic questions, technical experience assessments, and Likert-scale evaluation items addressing integration complexity, framework maturity, documentation quality, community support availability, and deployment readiness perceptions.

Survey Distribution Channels Figures A.14 through A.19 document the survey dissemination strategy across six professional platforms, ensuring diverse sampling across neuromorphic specialists (Open Neuromorphic, Neuromorphs), academic researchers (ResearchGate), and industry practitioners (LinkedIn, Slack, Reddit). This multi-channel approach addresses potential sampling bias while capturing perspectives from both SNN-experienced and conventional ML practitioners.

A.3 Google Form Survey - Results

```
(.venv) (base) bth@rparada-NUC7i7BNH:~/Projects/SE_thesis_snn$ python run_pipeline.py

=====
SNN LOG ANOMALY DETECTION - PRODUCTION PIPELINE
Features: Adaptive Architecture | Threshold Tuning | Complete Metrics
=====

=====
ANALYZING SOFTWARE ENGINEERING METRICS
=====

Analyzing SNN...
LoC: 3088, Comments: 134, Complexity: 3.32

Analyzing Transformer...
LoC: 2414, Comments: 90, Complexity: 2.46

Analyzing IsolationForest...
LoC: 2414, Comments: 90, Complexity: 2.46

=====

=====
BATCH MODE - PROCESSING MULTIPLE DATASETS
=====

=====
DATASET DISCOVERY
=====
Searching in: /home/bth/Projects/SE_thesis_snn/data
Found 16 dataset(s):
  1. Android_2k.log_structured_corrected.csv (435.0 KB)
  2. Apache_2k.log_structured_corrected.csv (250.8 KB)
  3. BGL_2k.log_structured_corrected.csv (412.7 KB)
  4. HDFS_2k.log_structured_corrected.csv (382.5 KB)
  5. HPC_2k.log_structured_corrected.csv (210.3 KB)
  6. Hadoop_2k.log_structured_corrected.csv (505.2 KB)
  7. HealthApp_2k.log_structured_corrected.csv (274.4 KB)
  8. Linux_2k.log_structured_corrected.csv (315.6 KB)
  9. Mac_2k.log_structured_corrected.csv (481.6 KB)
 10. OpenSSH_2k.log_structured_corrected.csv (347.3 KB)
 11. OpenStack_2k.log_structured_corrected.csv (695.8 KB)
 12. Proxifier_2k.log_structured_corrected.csv (325.3 KB)
 13. Spark_2k.log_structured_corrected.csv (293.6 KB)
 14. Thunderbird_2k.log_structured_corrected.csv (426.4 KB)
 15. Windows_2k.log_structured_corrected.csv (390.4 KB)
 16. Zookeeper_2k.log_structured_corrected.csv (361.0 KB)

=====
PIPELINE: Android_2k.log_structured_corrected
=====

[STAGE 1/6] Data Loading...

=====
LOADING: Android_2k.log_structured_corrected.csv
=====

[TEMPLATE ANALYSIS]
  Template file: Android_2k.log_templates_corrected.csv
  Template rows: 158
  Using columns: EventId='EventId', Template='EventTemplate'
  Anomalous EventIDs found: 9
  Labels: 1972 normal (98.6%), 28 anomaly (1.4%)
  Loaded: 2000 entries, 11 columns
```

Figure A.7: Console output showing pipeline execution start within Docker container. The orchestration layer (`run_pipeline.py`) initializes by loading `config.yaml`, detecting 16 Loghub datasets, and beginning Stage 1 (Data Ingestion) with progress indicators showing CSV parsing, schema detection, tokenization, and sliding window creation. Status messages display dataset statistics including log count, vocabulary size, and anomaly percentage.

```

[STAGE 6/6] Evaluation...

=====
COMPLETE COMPARISON - ALL METRICS
=====

Metric                                SNN      Transformer      IsoForest
-----
Accuracy (%)                          82.71      82.96      19.05
Balanced Acc (%)                      77.91      N/A      N/A
Precision (%)                         87.33      81.56      5.00
Recall (%)                           88.85      98.61      0.70
F1-Score (%)                         88.08      89.27      1.22
Parameters                           11,074      399,490      N/A
Latency (ms)                         0.0324      0.0492      0.0422
CO2 (g)                              0.0000      0.0000      0.0000
=====

=====
Complete: Zookeeper_2k.log_structured_corrected
=====

=====
BATCH PROCESSING SUMMARY
=====
Total evaluated:    15
Successful:         14
Zero detection:     1
Good F1 (>65%):    9
Average F1:         69.07%
Average Recall:     74.80%
Average Precision:  82.35%
=====

Summary saved: logs/run_20251112_192605/pipeline_summary.json
SE Metrics saved: logs/run_20251112_192605/software_engineering_metrics.json

=====
PIPELINE COMPLETE
=====

```

Figure A.8: Pipeline execution completion output displaying comprehensive comparative evaluation results. The terminal shows side-by-side metrics for SNN (Rate/Temporal/Latency encodings), Transformer baseline, and Isolation Forest across four evaluation dimensions: (1) Detection Performance (accuracy, F1, precision, recall), (2) Computational Efficiency (latency, parameters), (3) Sustainability (CO_2 emissions), and (4) Software Engineering Quality (LoC, MI, CC). Results are serialized to `logs/pipeline_summary.json` and visualization scripts generate publication-quality plots.

The image shows a Google Form titled "SNN vs Traditional ML for Log Anomaly Detection". It is labeled "Section 1 of 5". The form includes a purpose statement: "Compare software engineering perspectives on integrating Spiking Neural Networks (SNNs) versus traditional ML for log monitoring." and an estimated time of "5 minutes". There are three main sections of questions, each with a title and a list of checkboxes. The first section is "Role" with options: Software Engineer, SRE, Academic, ML Engineer, DevOps/MLOps, and Other. The second section is "Years experience" with options: 0-2, 3-5, 6-10, and 10+. The third section is "ML models integrated into production" with options: 0, 1-2, 2, 3-5, and 6+. At the bottom, there is a navigation bar with "After section 1" and "Continue to next section".

Section 1 of 5

SNN vs Traditional ML for Log Anomaly Detection

B I U

Purpose: Compare software engineering perspectives on integrating Spiking Neural Networks (SNNs) versus traditional ML for log monitoring.
Time: 5 minutes

Role

- ☐ Software Engineer
- ☐ SRE
- ☐ Academic
- ☐ ML Engineer
- ☐ DevOps/MLOps
- ☐ Other:

Years experience

- ☐ 0-2
- ☐ 3-5
- ☐ 6-10
- ☐ 10+

ML models integrated into production

- ☐ 0
- ☐ 1-2
- ☐ 2
- ☐ 3-5
- ☐ 6+

After section 1 Continue to next section

Figure A.9: Survey introduction page presenting research context, informed consent disclosure, participant confidentiality assurances, and estimated completion time (8-10 minutes). The introduction explains that survey responses will inform SNN integration best practices and framework maturity assessments documented in Chapter 5.

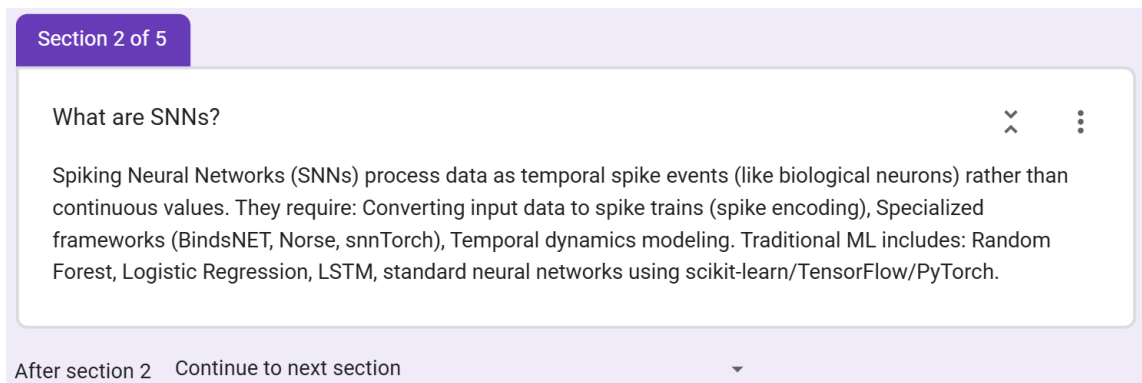


Figure A.10: Demographic and professional background questions capturing respondent role (researcher, practitioner, student), years of ML/DL experience, prior SNN exposure, and primary application domains. These variables enable stratified analysis of responses based on expertise level and domain specialization.

Section 3 of 5

Part A: SNN Integration for Log Anomaly Detection

Based on your professional judgment, rate each statement:
Scale: 1 = Strongly Disagree, 5 = Strongly Agree

A modular architecture (separate data loading, spike encoding, SNN model, evaluation) supports good maintainability

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Integrating SNNs is more complex than traditional ML due to spike encoding requirements

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Specialized SNN frameworks (BindsNET, Norse) create long-term maintenance concerns

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

SNNs can be effectively deployed using Docker and standard CI/CD pipelines

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Comprehensive documentation can make SNN integration accessible to practitioners

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

SNNs are production-ready for enterprise log monitoring systems

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Overall, SNN integration has acceptable software engineering quality for production use

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

After section 3 Continue to next section

Figure A.11: Technical experience assessment questions evaluating familiarity with neuromorphic computing concepts, SNN frameworks (SpikingJelly, Brian2, Norse, snnTorch), spike encoding strategies, and prior deployment experience in production systems. Responses establish baseline technical proficiency for interpreting subsequent evaluation items.

Section 4 of 5

Part B: Traditional ML for Log Anomaly Detection

Rate traditional ML (Random Forest, LSTM, standard neural nets) for the same aspects.
Scale: 1 = Strongly Disagree, 5 = Strongly Agree

A modular architecture supports good maintainability

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Integration complexity is manageable with standard preprocessing

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Standard ML frameworks (scikit-learn, TensorFlow, PyTorch) have low maintenance risk

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Traditional ML deploys easily with Docker and CI/CD pipelines

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Documentation for traditional ML integration is generally sufficient

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Traditional ML is production-ready for enterprise log monitoring

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

Overall, traditional ML has good software engineering quality

1 2 3 4 5

☐ ☐ ☐ ☐ ☐

After section 4 Continue to next section

Figure A.12: Framework maturity evaluation questions using 5-point Likert scales to assess perceptions of SpikingJelly documentation quality, API usability, debugging tool availability, and community support responsiveness. These items directly address RQ2 regarding best practices and framework ecosystem maturity.

The screenshot shows a Google Form titled "Section 5 of 5". The first section is titled "Your Expert Perspective" and includes the instruction "Please, answer the below questions with your own words". It contains three long-answer text questions:

- What engineering factors would make you consider using SNNs for log anomaly detection?
- What are the main engineering concerns about using SNNs versus traditional ML?
- Additional thoughts on SNN vs traditional ML integration:

The final section is a "Thank you!" message, followed by a survey code and a link to SurveyCircle. A red circle with the number "2" is visible in the bottom right corner of the form.

Figure A.13: Integration complexity assessment questions evaluating perceived difficulty of SNN integration compared to conventional ML frameworks, deployment complexity, monitoring requirements, and long-term maintainability concerns. Responses inform RQ1 regarding software engineering challenges and trade-offs.

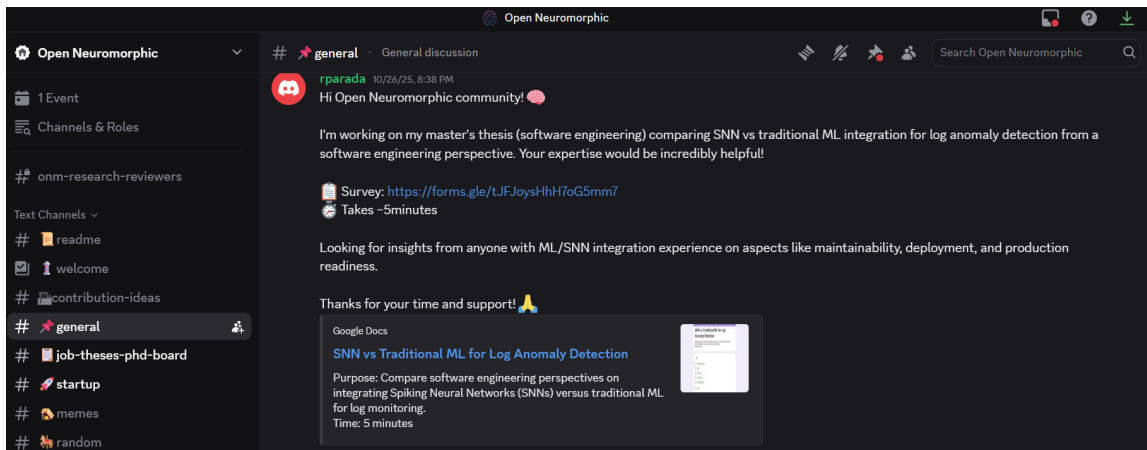


Figure A.14: Survey distribution via Open Neuromorphic (ONM) Discord server targeting neuromorphic computing specialists, SNN framework developers, and researchers actively working with spiking neural networks. The ONM community comprises over 2,000 members including academic researchers, industry practitioners, and neuromorphic hardware specialists. This channel provides access to practitioners with direct hands-on SNN implementation experience, ensuring survey responses reflect practical deployment challenges rather than purely theoretical perspectives.

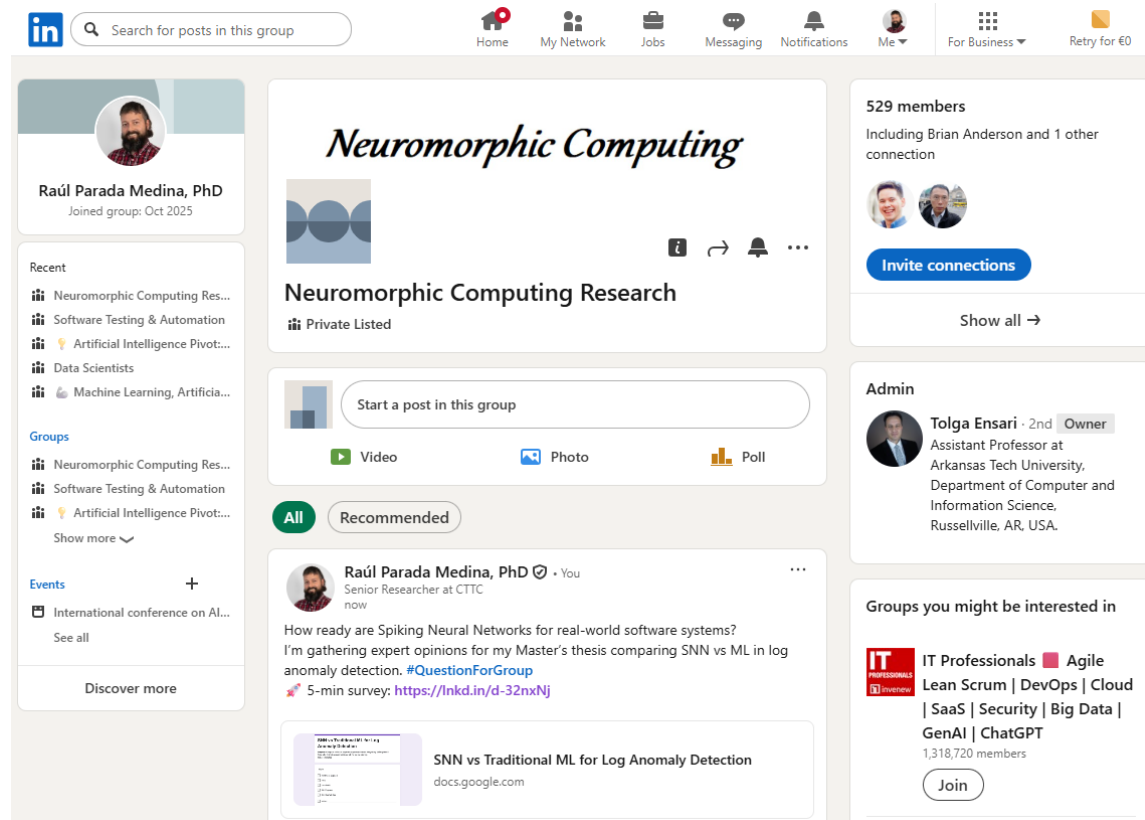


Figure A.15: Survey shared via LinkedIn professional network targeting ML engineers, MLOps practitioners, software architects, and DevOps professionals in the broader AI/ML industry. LinkedIn distribution ensures representation from enterprise practitioners evaluating SNNs for potential production deployment, capturing perspectives beyond academic research contexts. Relevant hashtags ([#MachineLearning](#), [#NeuromorphicComputing](#), [#MLOps](#), [#SoftwareEngineering](#)) maximize visibility across professional networks, enabling stratified sampling across experience levels and organizational contexts.

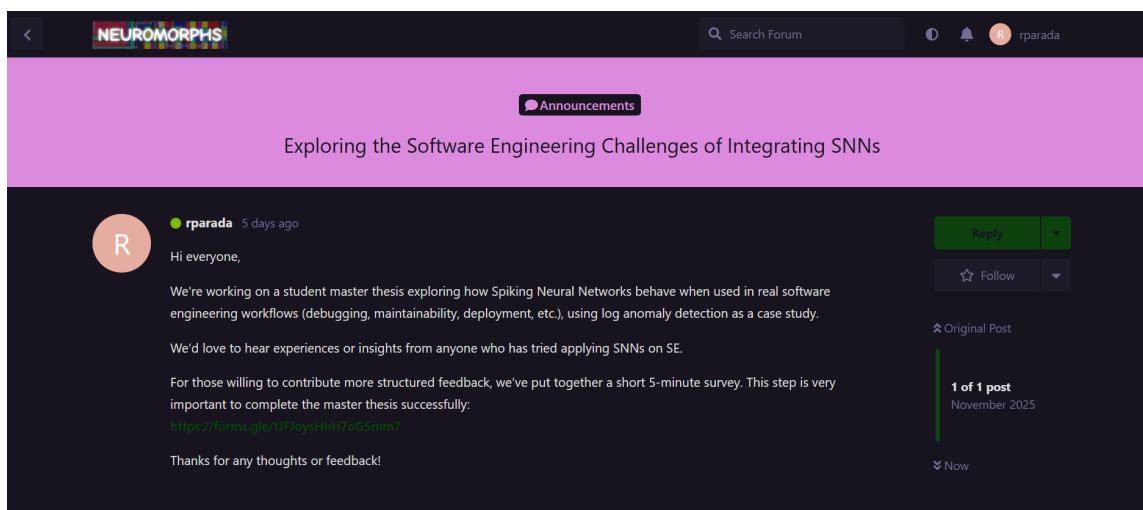
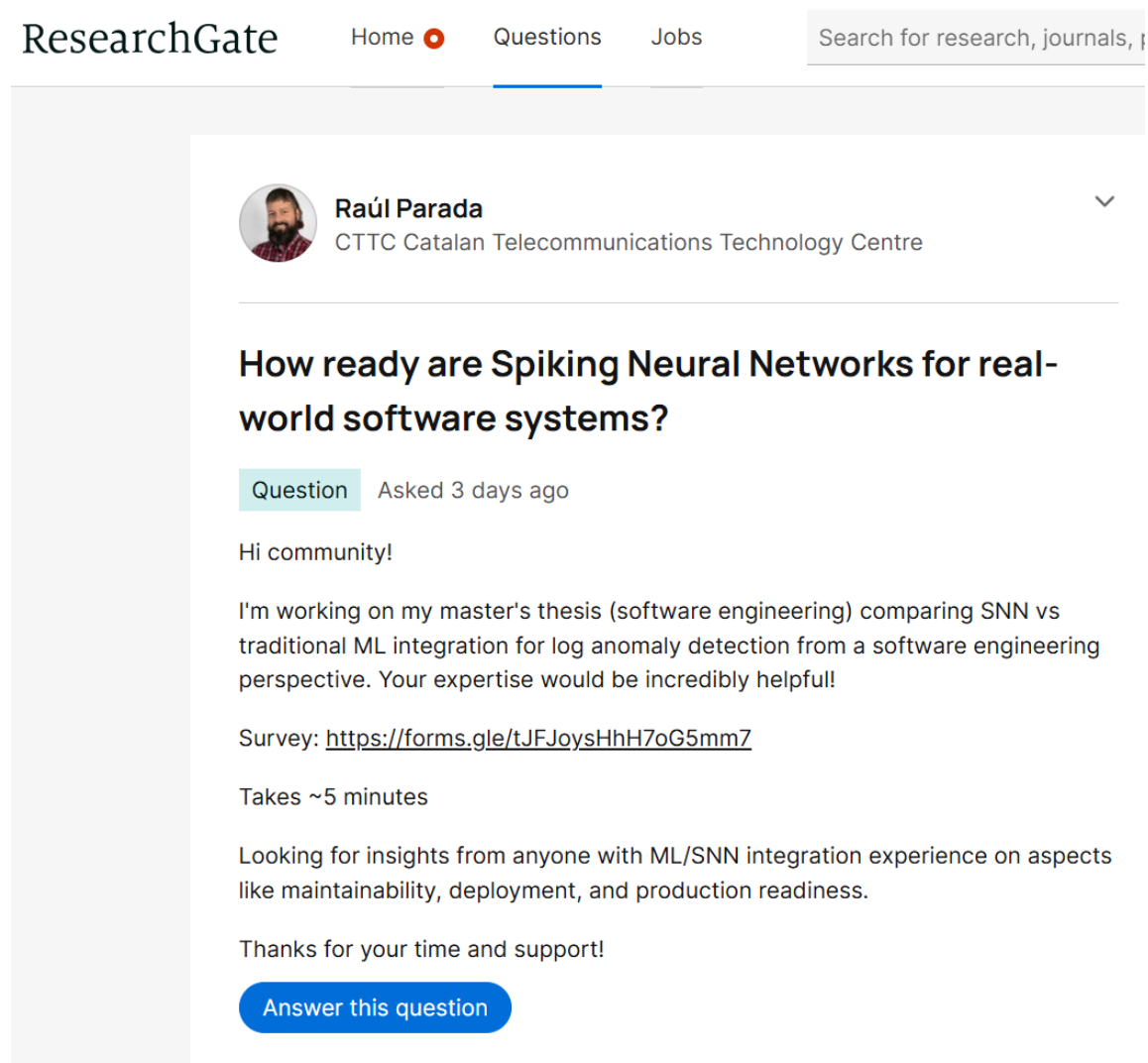


Figure A.16: Survey announcement posted to Neuromorphs community platform, an independent neuromorphic computing discussion forum and knowledge-sharing hub. This platform reaches neuromorphic enthusiasts, hardware designers, and algorithm researchers who may not actively participate in larger Discord communities. Distribution through multiple neuromorphic-focused channels (ONM Discord + Neuromorphs) ensures comprehensive coverage of specialized practitioner populations with varying community engagement preferences.



The screenshot shows the ResearchGate website interface. At the top, there is a navigation bar with the ResearchGate logo, links for Home, Questions, and Jobs, and a search bar. Below the navigation bar, the profile of Raúl Parada is displayed, including his profile picture and affiliation with CTTC Catalan Telecommunications Technology Centre. The main content of the post is a question titled "How ready are Spiking Neural Networks for real-world software systems?". The question is marked as "Asked 3 days ago". The text of the question reads: "Hi community! I'm working on my master's thesis (software engineering) comparing SNN vs traditional ML integration for log anomaly detection from a software engineering perspective. Your expertise would be incredibly helpful! Survey: <https://forms.gle/tJFJoysHhH7oG5mm7> Takes ~5 minutes Looking for insights from anyone with ML/SNN integration experience on aspects like maintainability, deployment, and production readiness. Thanks for your time and support!". At the bottom of the question text, there is a blue button that says "Answer this question".

ResearchGate

Home Questions Jobs

Search for research, journals, p

 **Raúl Parada**
CTTC Catalan Telecommunications Technology Centre

How ready are Spiking Neural Networks for real-world software systems?

Question Asked 3 days ago

Hi community!

I'm working on my master's thesis (software engineering) comparing SNN vs traditional ML integration for log anomaly detection from a software engineering perspective. Your expertise would be incredibly helpful!

Survey: <https://forms.gle/tJFJoysHhH7oG5mm7>

Takes ~5 minutes

Looking for insights from anyone with ML/SNN integration experience on aspects like maintainability, deployment, and production readiness.

Thanks for your time and support!

Answer this question

Figure A.17: Survey distributed through ResearchGate professional research network targeting academic researchers, PhD students, and postdoctoral fellows working on neuromorphic computing, anomaly detection, or ML systems engineering. ResearchGate's academic focus complements industry-oriented platforms (LinkedIn, Slack), enabling comparison between academic and industrial practitioner perspectives on SNN deployment readiness, framework maturity, and integration challenges. Direct researcher-to-researcher distribution enhances response credibility for qualitative feedback.

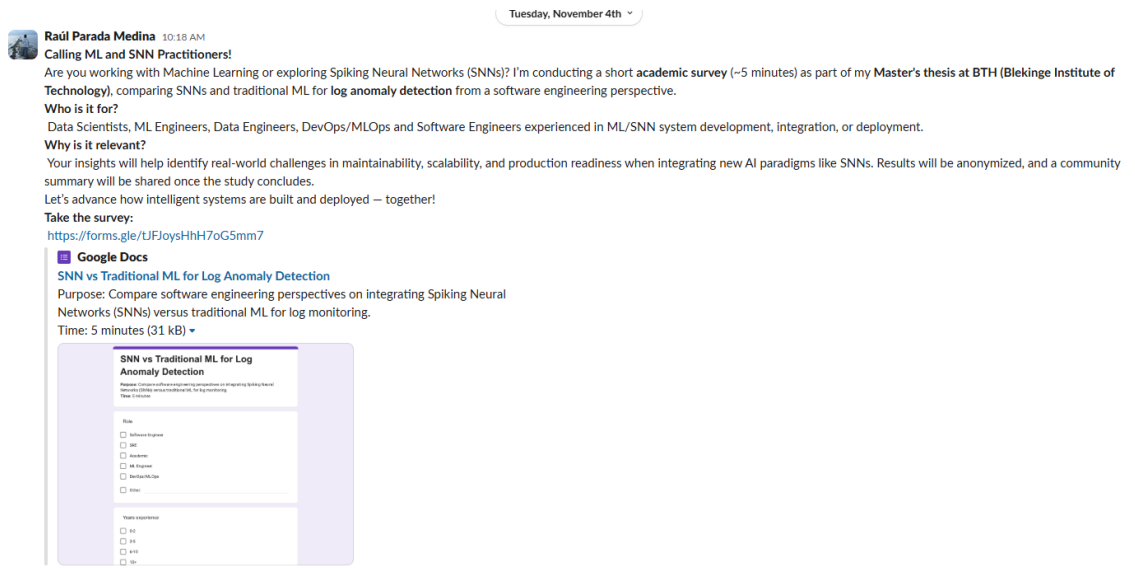


Figure A.18: Survey posted to MLOps Community Slack workspace (#research or #general channel) reaching DevOps engineers, site reliability engineers (SREs), and infrastructure practitioners responsible for deploying and maintaining production ML systems. This channel targets practitioners focused on operational concerns (monitoring, deployment automation, infrastructure costs, maintainability) rather than purely algorithmic performance, ensuring survey captures deployment-centric perspectives critical for addressing RQ1 (integration challenges) and RQ2 (best practices).

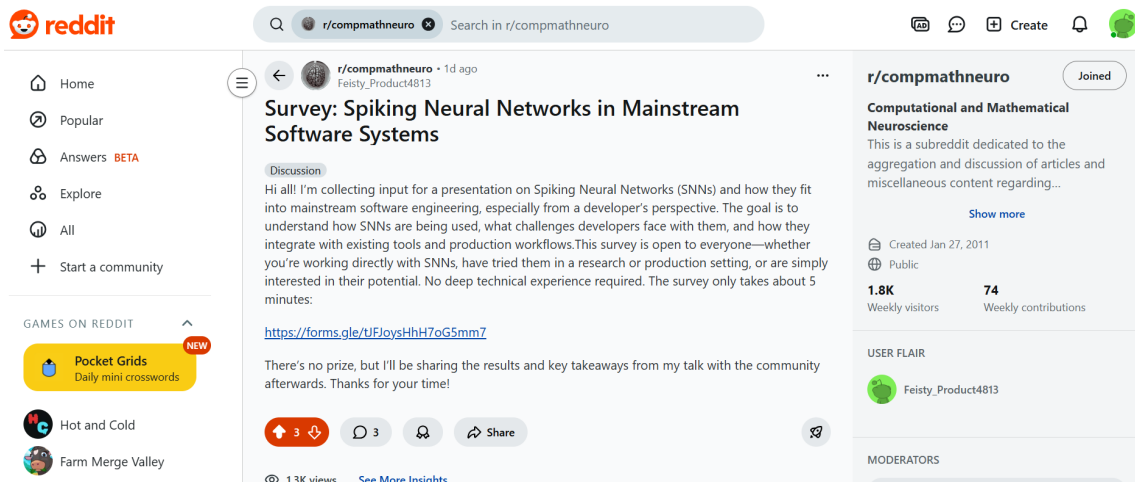


Figure A.19: Survey shared on Reddit machine learning communities (r/Machine-Learning, r/computervision, or r/neuroscience) reaching diverse ML practitioner population spanning hobbyists, students, industry engineers, and researchers. Reddit's broad demographic (2.7M+ members across ML subreddits) provides access to practitioners outside specialized neuromorphic circles, capturing perspectives from conventional ML engineers encountering SNN concepts potentially for the first time. Community voting mechanisms and open discussion threads enable public transparency regarding survey methodology and research objectives.

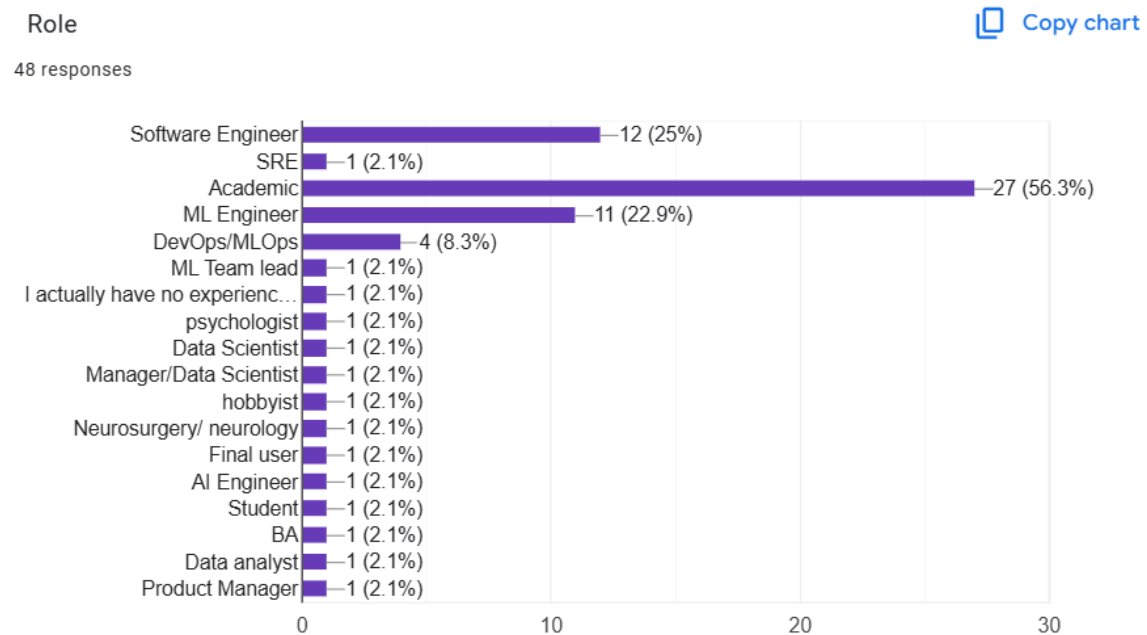


Figure A.20: Professional roles of survey participants: the sample is dominated by academics, software engineers, and ML engineers, with smaller groups from SRE, DevOps/MLOps, and other related roles.

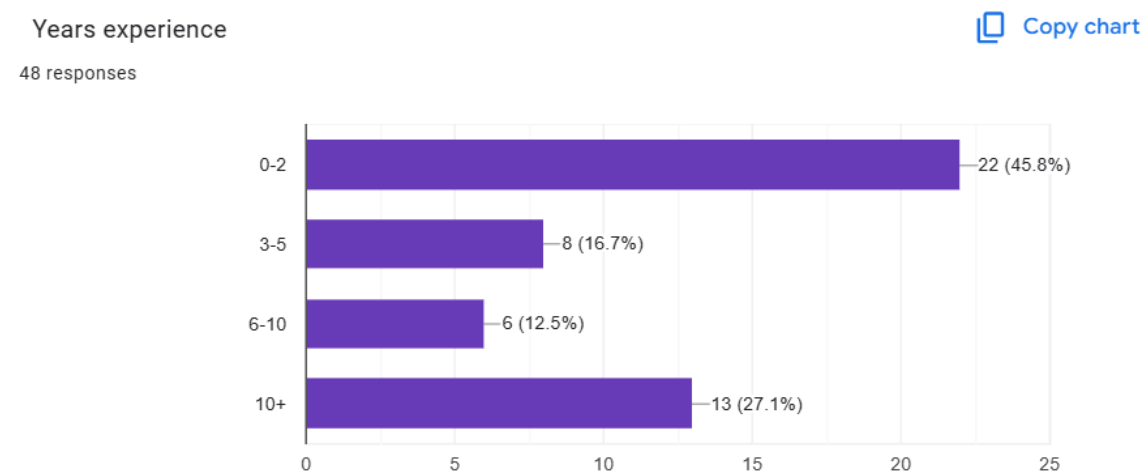


Figure A.21: Years of experience of survey participants: nearly half report 0–2 years, while over a quarter have 10+ years, indicating a mix of junior and highly experienced practitioners.

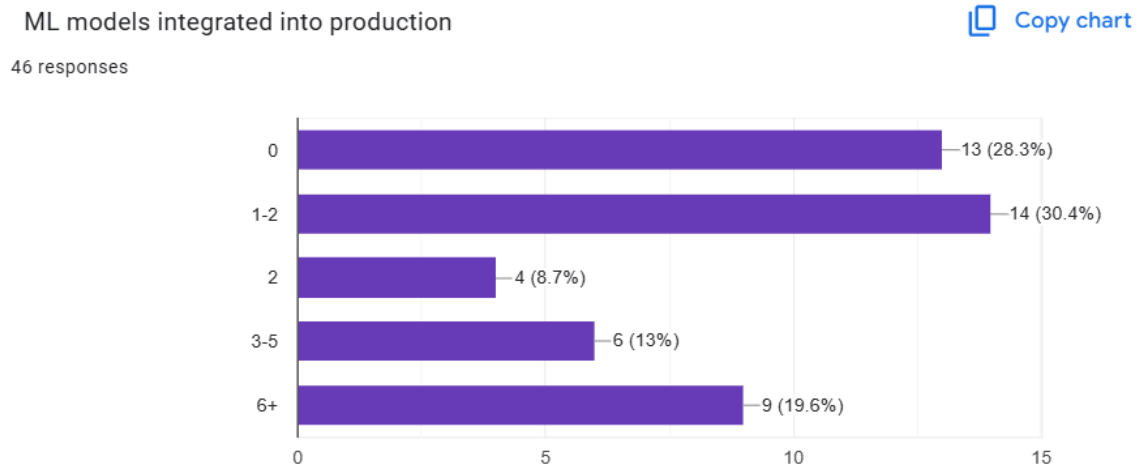


Figure A.22: Self-reported number of ML models respondents have integrated into production, showing a broad range of experience levels and confirming that many participants have direct exposure to deployment challenges.

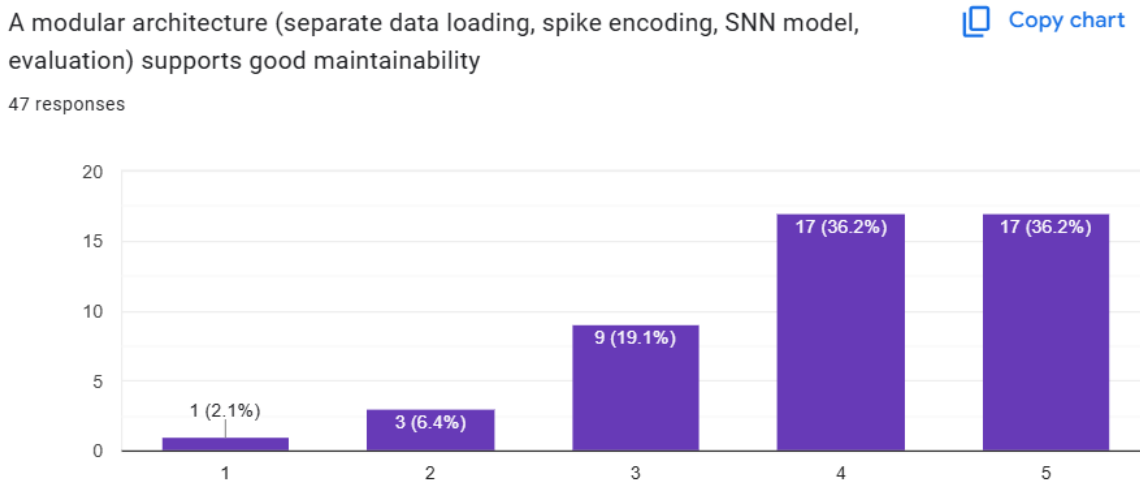


Figure A.23: Perceived impact of a modular architecture (separate data loading, spike encoding, SNN model, evaluation) on maintainability: the majority of practitioners rate the statement positively, supporting the architectural approach used in this thesis.

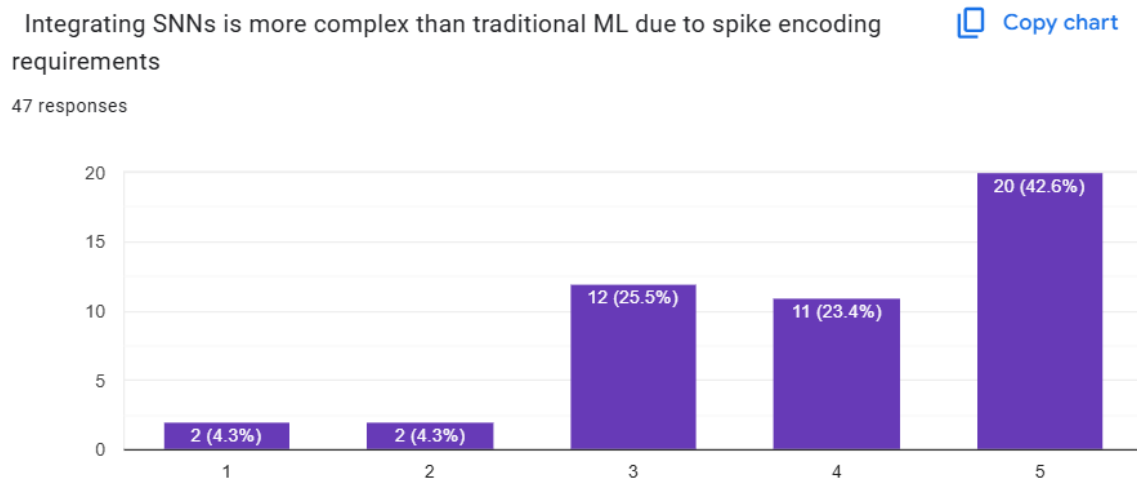


Figure A.24: Perceived integration complexity of SNNs compared to traditional ML due to spike encoding requirements: most respondents agree that SNNs are more complex to integrate.

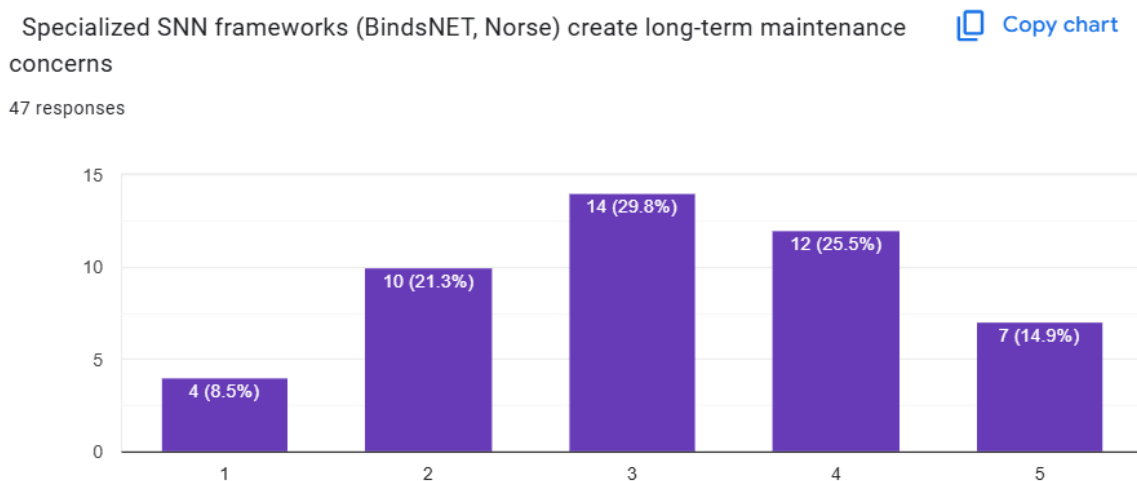


Figure A.25: Perception that specialised SNN frameworks (e.g., BindsNET, Norse) create long-term maintenance concerns: many respondents express moderate to strong concern about relying on niche neuromorphic libraries.

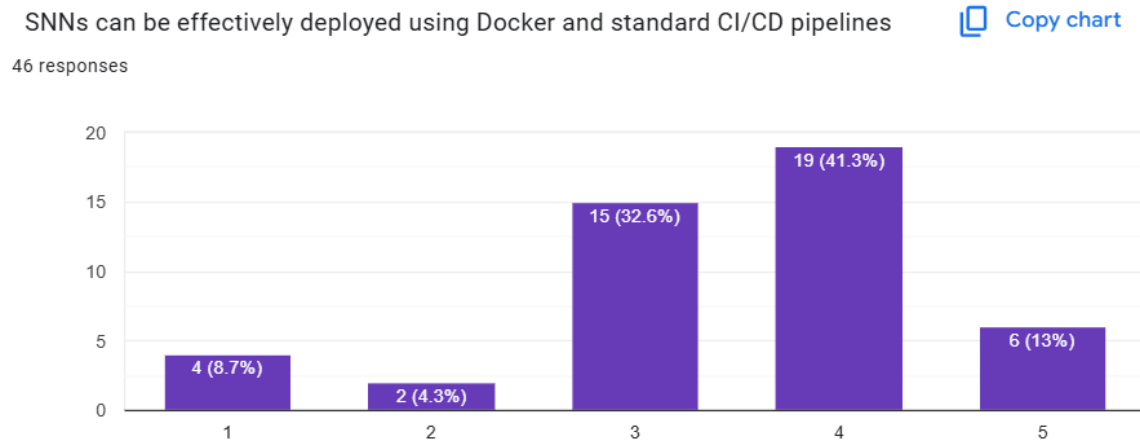


Figure A.26: Perceived feasibility of deploying SNNs using Docker and standard CI/CD pipelines: most respondents agree that conventional DevOps tooling is sufficient, although a minority remains sceptical.

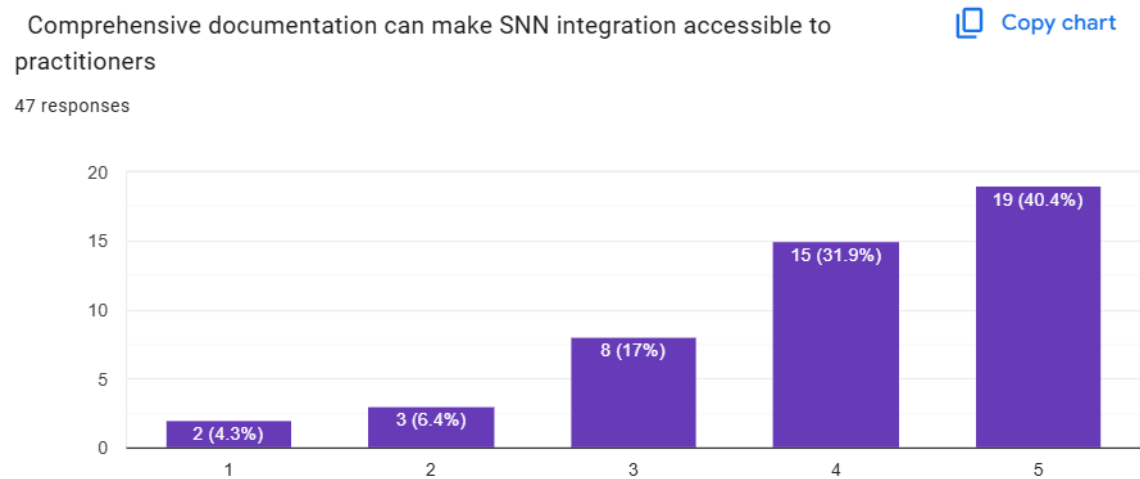


Figure A.27: Likert-scale responses to the statement “Comprehensive documentation can make SNN integration accessible to practitioners”: most respondents agree (4) or strongly agree (5), highlighting documentation as a key mitigation for ecosystem immaturity.

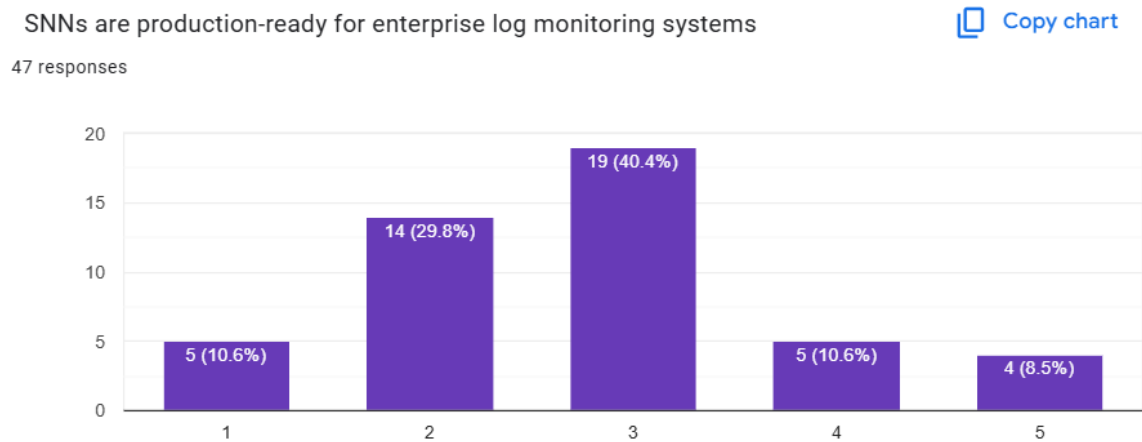


Figure A.28: Perception of SNN production-readiness for enterprise log monitoring systems: responses cluster around the neutral midpoint, with relatively few respondents strongly endorsing immediate production use.

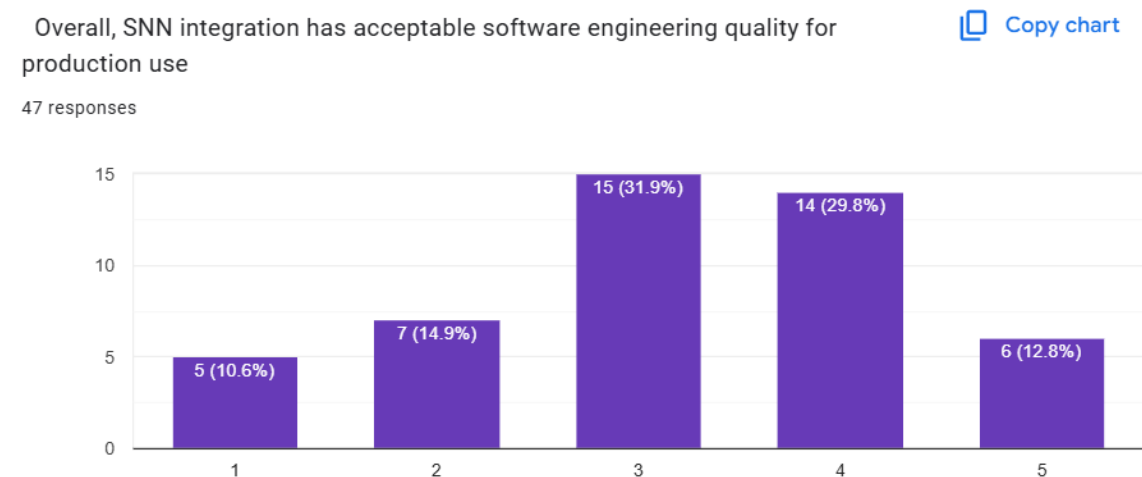


Figure A.29: Overall perception of SNN integration quality for production use: most respondents remain neutral or only somewhat positive, indicating that SNNs are not yet broadly seen as production-ready components.

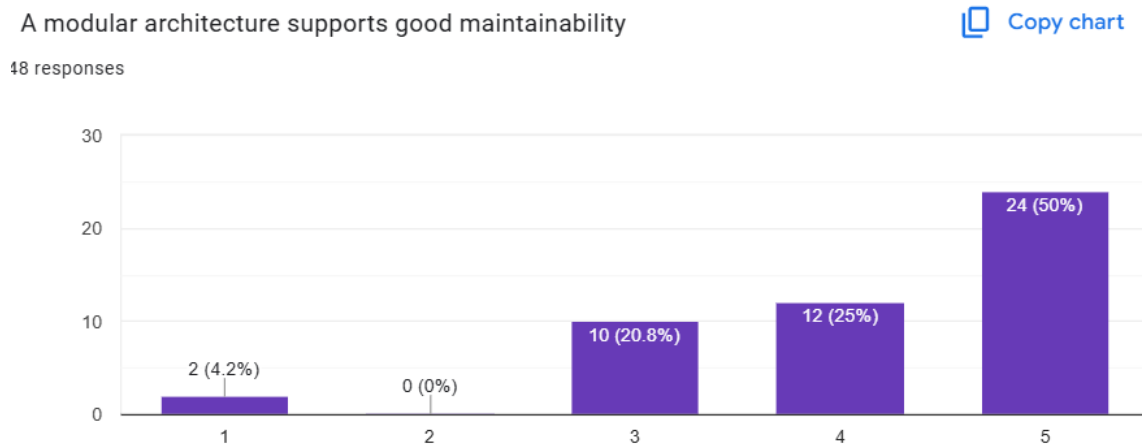


Figure A.30: Practitioner ratings for the statement “A modular architecture supports good maintainability”: the majority agree or strongly agree, reinforcing the design choice of separating data loading, encoding, models, and evaluation.

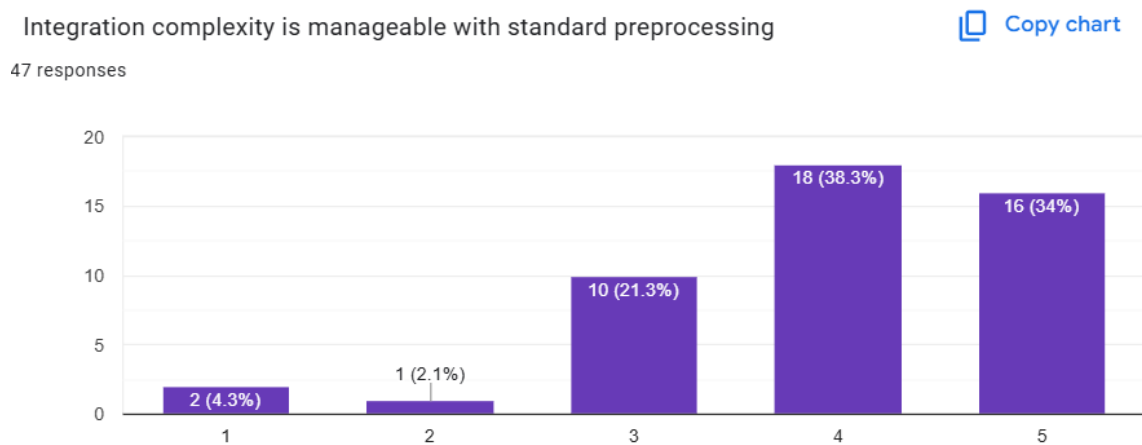


Figure A.31: Perception that integration complexity is manageable with standard preprocessing: most respondents agree or strongly agree, indicating that traditional ML integrations are viewed as tractable with existing data pipelines.

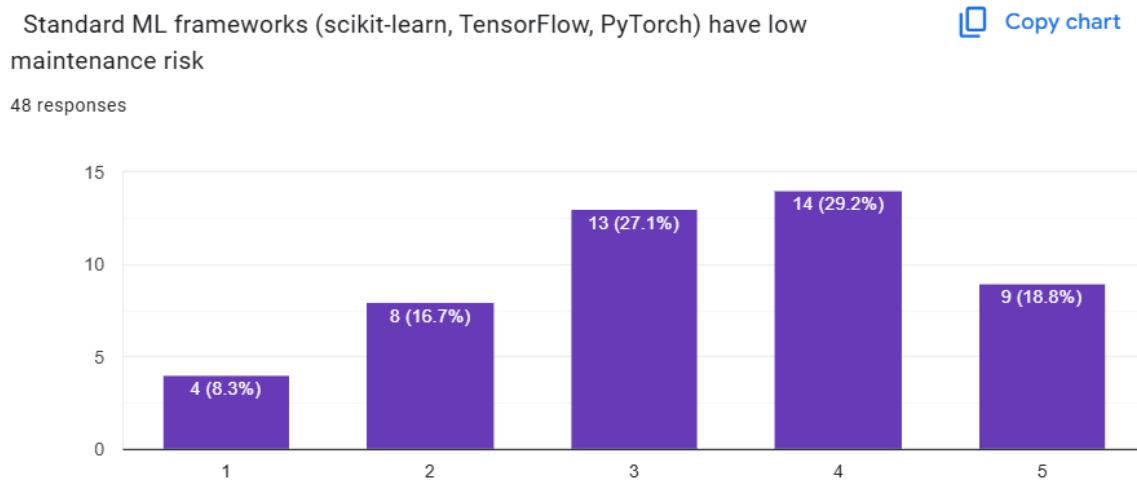


Figure A.32: Perception that standard ML frameworks (such as scikit-learn, TensorFlow, and PyTorch) have low maintenance risk: most respondents agree or strongly agree, reflecting confidence in their long-term stability and support.

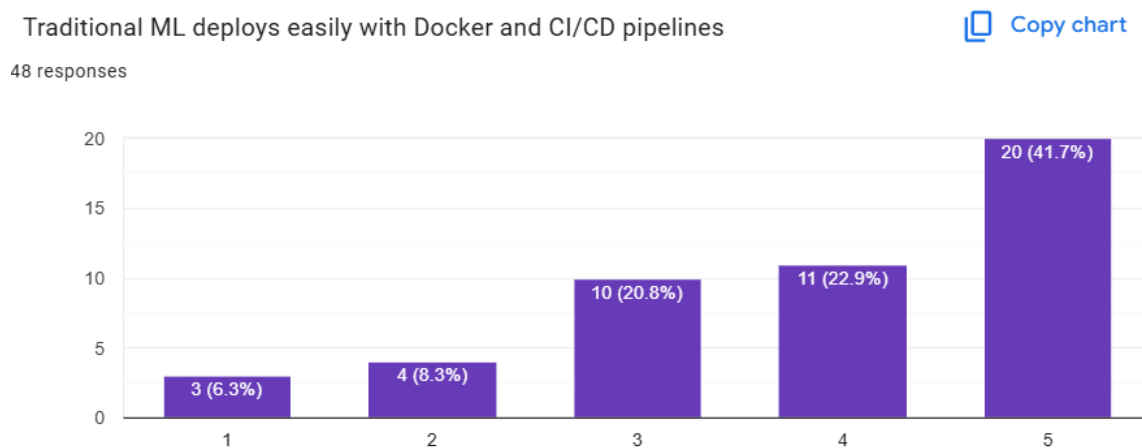


Figure A.33: Perception that traditional ML deploys easily with Docker and CI/CD pipelines: strong agreement dominates, confirming that conventional models fit well into existing DevOps practices.

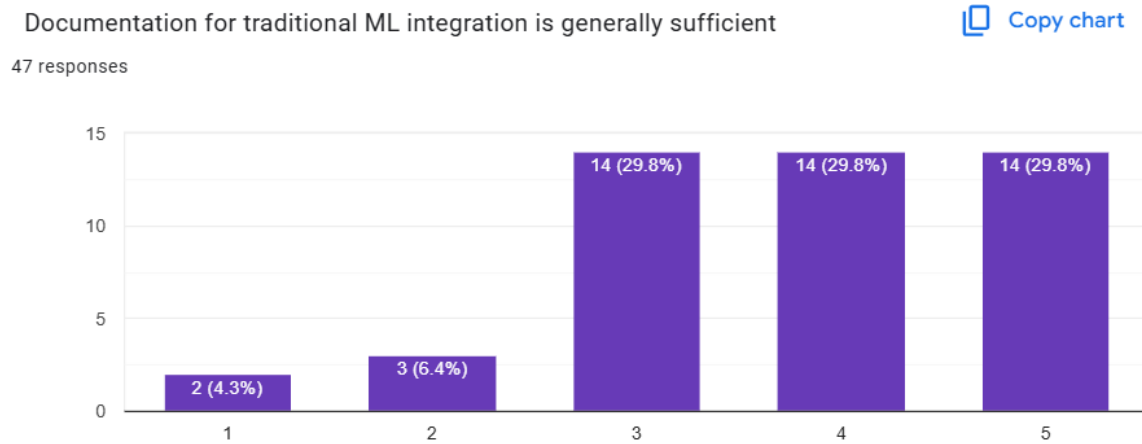


Figure A.34: Perception that documentation for traditional ML integration is generally sufficient: roughly two thirds of respondents rate the documentation as good or very good, supporting smoother onboarding and maintenance.

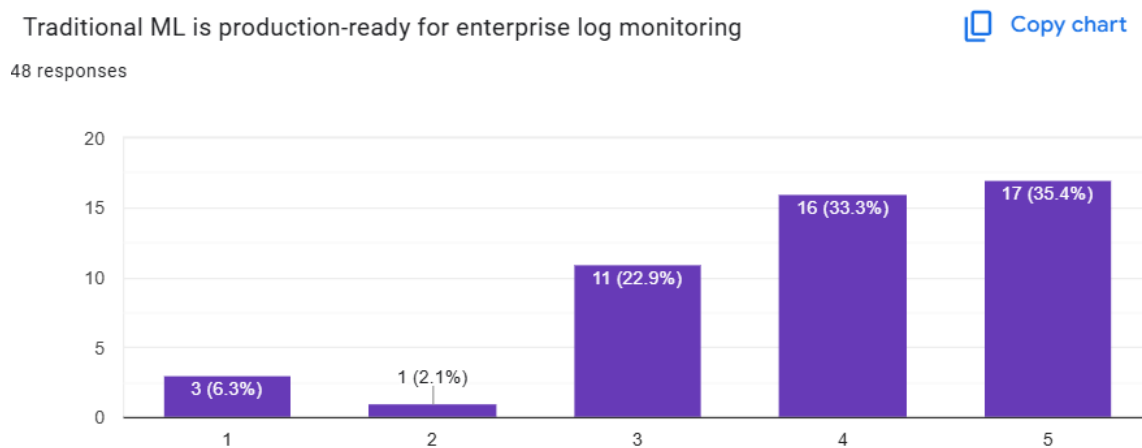


Figure A.35: Perception that traditional ML is production-ready for enterprise log monitoring: strong agreement dominates, reflecting high confidence in the maturity and robustness of conventional approaches.

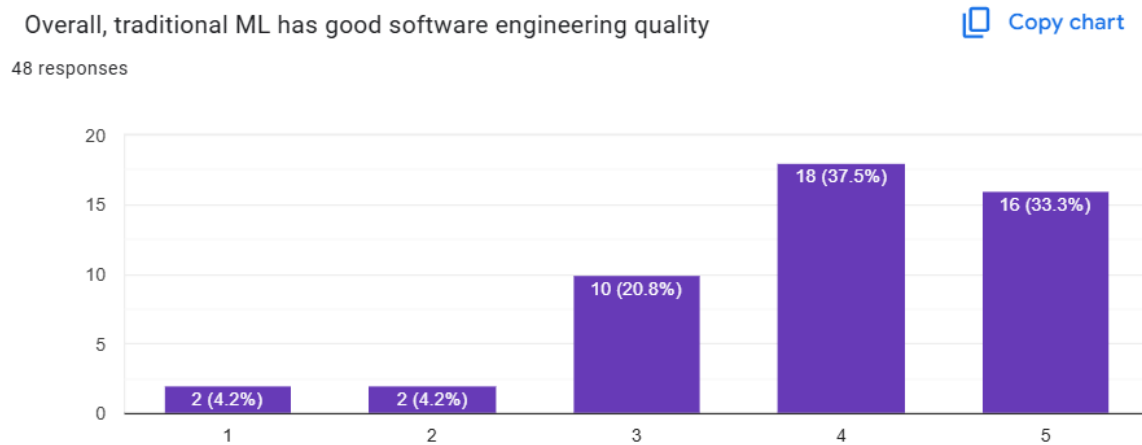


Figure A.36: Overall assessment of traditional ML software engineering quality: the vast majority of respondents agree that traditional ML components exhibit good engineering properties for production use.

